From Imperative to Reactive Time-Synchronized Systems: An NTP Case Study

Deepali Banka
School of Electronics Engineering (SENSE)
Vellore Institute of Technology, Chennai, India
deepali.banka2023@vitstudent.ac.in

Anupam Chattopadhyay
College of Computing and Data Science
Nanyang Technological University, Singapore
anupam@ntu.edu.sg

arXiv:submit/8305651 [cs.DC] 30 Oct 2026

***Abstract*—This paper compares time-synchronized system design using Embedded C and Lingua Franca (LF). Leveraging the Network Time Protocol (NTP) as a case study, we evaluate how these paradigms manage clock skew, concurrency, and temporal semantics. Within Cyber-Physical Systems (CPS), where physical reliability depends on precise computational timing, time-synchronization serves as the fundamental backbone for aligning distributed logic with real-world time. While Embedded C offers minimal resource overhead, its imperative model requires manual timing and concurrency management. This imposes a significant cognitive burden and often introduces an additional task of managing non-deterministic jitter. Conversely, LF's reactor-oriented mechanism treats time as a first-class construct, ensuring deterministic execution through semantic elevation. By evaluating the infrastructure and manual programming cost of achieving temporal correctness, we demonstrate that LF provides a superior abstraction for complex CPS. We conclude that for systems requiring verifiable coordination, the reactor model's structural advantages, such as native logical tag ordering and lock-free fault signaling, fully justify its larger software footprint. This trade-off is particularly vindicated under adverse network conditions like path asymmetry, out-of-order packet delivery, and partial network failure, where LF natively prevents the thread blocking, data races, and scheduling jitter that plague manual, imperative implementations.**

***Index Terms*—Network Time Protocol (NTP), Cyber-Physical Systems (CPS), Lingua Franca, Embedded C, Reactor Model, Time Synchronization**

I. Introduction

MODERN CPS, such as autonomous vehicles and smart grids, rely on precise coordination between computational logic and real-world time [1]. Software "correctness" is defined by both numerical accuracy and temporal precision; therefore, time synchronization serves as the structural foundation for aligning distributed components. While Embedded C offers high execution efficiency, its imperative paradigm lacks native constructs for time and concurrency [2]. Developers must manually manage low-level primitives to approximate temporal behavior. Conversely, LF utilizes a reactor oriented paradigm to provide a higher level of architectural abstraction. By treating time as a first-class language construct, LF shifts coordination from the programmer to the compiler, ensuring deterministic execution by design. We utilize NTP as a case study to evaluate these paradigms. NTP's operation requires precise clock skew measurement and robust concurrency, directly challenging the temporal limits of both models.

The contributions of this study include a paradigm comparison wherein we evaluate how imperative and reactor-oriented models handle temporal semantics and concurrency, an implementation benchmarking to assess the development effort required to achieve deterministic timing, and a software engineering overhead analysis quantifying the extra programming effort required to manually build features in C that are native to the LF compiler.

II. Background and Related Work

To ground our paradigm comparison, this section outlines the operational fundamentals of NTP as our evaluation testbed and the deterministic timing semantics of the Reactor Model in LF. We then review the current research on concurrent execution and temporal predictability in CPS.

A. NTP Fundamentals

NTP is the standard for synchronizing clocks across distributed networks to a single time source. As per the client-server architecture established by *Mills* [3], synchronization is achieved through a four timestamp exchange (T_1 – T_4) representing request and response transitions. The complete technical details and core operational algorithms are formally specified in the Network Time Protocol Version 4 document [4].

By calculating the clock offset, the primary metric for measuring skew, a client can adjust its local clock to account for network delay. This process serves as our canonical testbed, as it requires precise management of concurrent network events and strict temporal accuracy.

B. The Reactor Model and Lingua Franca

LF is a polyglot coordination language designed to bring determinism to concurrent and distributed systems via the Reactor Model. Unlike traditional actor models that are inherently asynchronous, as described by *Lohstroh et al.* [5], reactors are components that react to discrete events at specific points in logical time. Key components include:

- *Reactors*: High-level components containing encapsulated state, configuration parameters, and an ordered sequence of reactions.

- *Reactions*: Atomic procedures triggered by discrete events, such as actions, timers, or ports, that produce logically simultaneous outputs.
- *Logical Actions*: Event triggers scheduled at fixed logical intervals to guarantee precise, deterministic scheduling that is often absent in imperative models.

As a polyglot framework, LF allows developers to write core logic in target languages like Python and C++ while the LF compiler manages the high-level temporal semantics and coordination.

C. Related Work and the Imperative Gap

Current research in CPS emphasizes the necessity of "time-aware" programming. In traditional Embedded C, programmers must manage concurrency using low-level primitives like usleep() or POSIX threads, which often lead to non-deterministic interleaving and race conditions. While synchronous languages like *Esterel* [6] introduced formal timing, they often struggle with the nondeterministic latencies of network protocols. LF bridges this gap by wrapping imperative code in a deterministic container. This study builds upon the foundational runtime evaluation work by evaluating LF's effectiveness specifically for network synchronization tasks [7] compared to manual imperative implementations.

III. Implementation Methodology

This study evaluates the architectural transition from an imperative to a reactor-oriented paradigm by implementing a clock-synchronization system based on the Network Time Protocol (NTP). Our implementation is structured around the standard four-timestamp exchange required to measure clock skew as detailed below.

A. Coordination Logic and Calculation

Both implementations follow the standard NTP exchange to determine an Offset between the client and the server. This process requires capturing four distinct transition points:

- T_1: Request departure from the Client
- T_2: Request arrival at the Server
- T_3: Response departure from the Server
- T_4: Response arrival back at the Client

The Offset is calculated using the following formula:

$$\text{Offset} = \frac{(T_2 - T_1) + (T_3 - T_4)}{2}$$

The efficiency of capturing these timestamps is used to evaluate how each paradigm manages concurrent events and temporal precision.

B. Architectural Flow Comparison

In C, the synchronization exchange is a "blocking sequence". The program captures T1, sends the request, and then idles while waiting for the network response to capture T4 (see Figure 1). In LF, the exchange is a "reactive loop". The Client

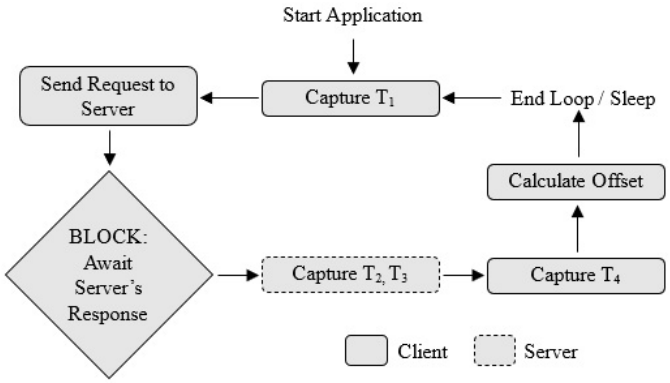


Figure 1: Blocking (Embedded C) Sequential Flow

reactor initiates T1 and then yields control to the runtime, re-activating only when the response arrives at its port to capture T4 (see Figure 2).

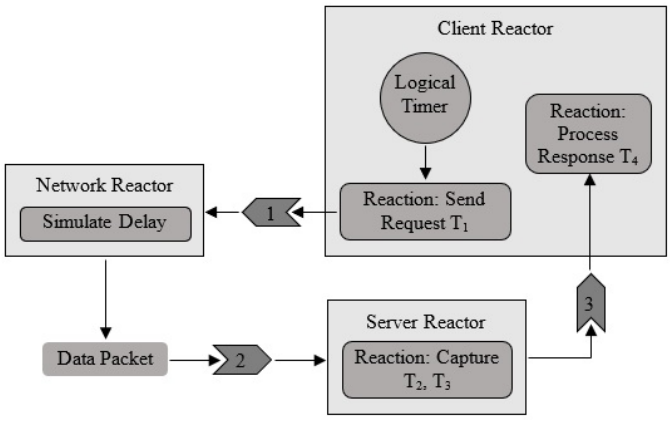


Figure 2: Reactive (LF) Event Graph

C. Execution Logic

The two implementations represent fundamentally different approaches to system coordination.

The imperative Embedded C approach is characterized by sequential, blocking execution where the system follows a rigid, linear control flow and network operations halt the entire process. In this model, timing is an implicit property of execution speed and manual sleep calls rather than a formal guarantee. The architecture exhibits tight coupling, meaning that application logic, network I/O, and timing are intertwined in a single monolithic structure. Since it is single-threaded by default, managing concurrent tasks requires manual (and often non-deterministic) integration of threading libraries or a real-time operating system.

Conversely, the reactive LF approach is characterized by event-driven, non-blocking execution, allowing the system to react to discrete events so that the CPU remains active or yields control without halting the application. Within this framework, timing is explicit and the temporal behavior is governed by logical actions and timers, providing a first-class definition of time. The architecture relies on loose coupling via ports, utilizing modular Reactors that communicate only through timestamped

Scenario	Embedded C (Imperative)	LF (Reactive)
Network Timeout	**System Block** – Thread hangs indefinitely, manual timeouts should be implemented	**Deterministic Timeout** - Timeout reaction registered and triggered automatically by the runtime
Component Crash	**System Failure** – Monolithic flow halts the entire application	**Fault Containment** – Errors strictly isolated within reactor boundaries, allowing rest of the system to function
Timing Violation	**Silent Error** – Jitters silently incorporated into the clock offset calculation	**Logical Consistency** – Lags managed by the scheduler deterministically using logical runtime semantics
Error Recovery	**Manual and Fragile** – Requires complex, manual state resetting and thread-safety variables	**Structured and Clean** – Reinvocation follows native event ordering and execution semantics
Debugging	**Non-Deterministic** – Hard to trace race conditions	**Deterministic** – Allows repeatable execution traces

Table 1: Robustness and Error Handling Scenarios

ports to isolate logic from the underlying coordination. The system features built-in concurrency because the LF runtime automatically identifies and executes independent reactions in parallel, ensuring deterministic results without manual thread management.

IV. Comparative Analysis

This section evaluates the practical impact of the architectural transition by comparing the Embedded C and LF implementations across various key dimensions. To ensure a fair comparison, both implementations were evaluated in a controlled environment. Intel i5 host running Ubuntu 24.04 was used as the platform. GCC 13.3.0 for the C target and LF 0.11.x utilizing the Python target were invoked, respectively. A localhost loopback simulation was used to isolate paradigm specific behavior from external network noise.

A. Robustness and Error Handling

The architectural transition shifts how the system maintains reliability under adverse conditions. In Embedded C, robustness is an emergent property of the programmer's manual skill, requiring low-level, error-prone thread and socket management. Conversely, Lingua Franca elevates robustness to a structural guarantee by managing execution safety natively within its runtime scheduler.

As summarized in Table 1, this superior abstraction is demonstrated across critical operational scenarios. Instead of relying on manual polling loops and blocking socket reads that can hang client threads indefinitely, LF handles network timeouts and failures deterministically via automated runtime triggers. Furthermore, LF provides strict fault containment by isolating errors within reactor boundaries, preventing a single component crash from halting the entire monolithic application. This model also preserves logical timing consistency and facilitates clean error recovery by managing scheduling lags deterministically using logical timestamps, avoiding silent physical OS jitter and fragile, shared-memory thread mutexes typical of imperative designs.

B. Timing Semantics and Deterministic Execution

The core distinction lies in the treatment of time, contrasting implicit physical time in C with explicit logical time in LF.

Under the implicit Embedded C approach, execution is governed by the OS scheduler, making the system prone to OS-induced jitter where any scheduling delay is baked into the NTP timestamp, thereby compromising the offset accuracy. Conversely, within the explicit Lingua Franca framework, reactions are ordered by logical timestamps, ensuring repeatable and coordinated timing. Its causality enforcement guarantees that effects never precede causes in logical time, providing a verifiable execution trace [8].

C. Implementation Efficiency and Effort

The transition from an imperative to a reactor oriented paradigm is best quantified by comparing the Total System Code (Client, Server and Network) against the infrastructure required for deterministic execution. Refer Table 2.

Metric	Embedded C	LF
Programmer Written Logic	∼180 LoC	∼156 LoC
Coordination Infrastructure	∼8,000 LoC	Native
Total System Footprint	∼180 LoC	∼20,000 LoC
Cyclomatic Complexity	High (Nested Logic)	Low (Flat Reactions)
Manual Effort	High (Infrastructure + Logic)	Low (Logic Only)

Table 2: Quantitative Implementation Metrics

When evaluated as a complete system, the Embedded C implementation (∼180 LoC) is more verbose than the unified Lingua Franca (LF) model (∼156 LoC). Despite its higher line count, the C implementation remains architecturally "incomplete" because it offloads all coordination and safety responsibilities to the programmer's manual skill.

The C implementation suffers from high cyclomatic complexity due to the need to manually manage nested network states, process synchronization, and timing loops across three fragmented programs. Conversely, the LF developer achieves a complete, coordinated system within a single, declarative file where the logic remains modular and flat.

This architectural shift effectively transfers the implementation burden from the human to the machine. While the LF compiler manages ∼20,000 lines of generated infrastructure to provide deterministic guarantees natively, a C developer would face an estimated 8,000 line infrastructure gap to reach a functional equivalent.

D. Software Engineering Overhead

As mentioned in the previous section, replicating LF's deterministic features in C requires significant extra programming effort. A functional equivalent of the reactor runtime in C

demands the manual construction of scheduling and timing via a manual event queue, deterministic scheduler, and action infrastructure. It also requires architectural logic through logical time representation, dependency graph management, and a port abstraction layer. Finally, it necessitates safety and reliability features, specifically determinism guards and built-in error handling mechanisms.

Our analysis indicates that a robust implementation of these features in C would exceed $8,000$ lines of manual code. In contrast, the LF compiler automatically generates this runtime support, providing multi-reactor concurrency, logical timestamp ordering, federated execution, advanced scheduling semantics, and causality enforcement to prevent out-of-order execution.

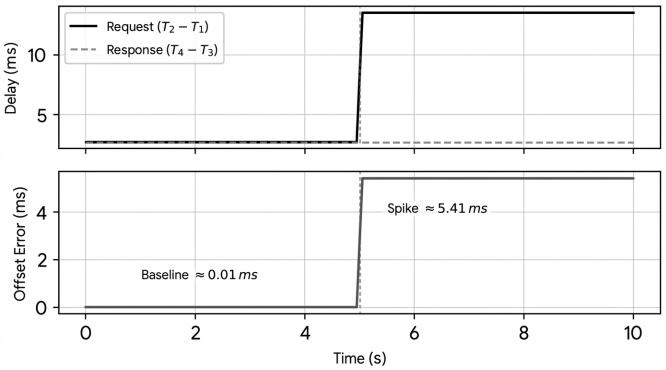


Figure 3: NTP Fluctuations under Congestion Asymmetry

V. Empirical Evaluation Under Network Variability

To quantify the operational resilience, clock accuracy and timing precision of both paradigms under real-world constraints, we subjected the respective synchronization infrastructures to a series of stressful network environments. These stress tests expose the critical architectural trade-offs between the native temporal orchestration of the reactor model and the imperative concurrency mechanisms of traditional Embedded C.

A. Bandwidth Throttling due to Network Congestion

We introduced a dynamic congestion factor into the request leg of the network to evaluate how both paradigms handle sudden link throttling mid-simulation. Standard NTP offset calculations mathematically assume that network delay is symmetric (that the time it takes for a request to travel to the server equals the time it takes for the response to return).

$$\text{Offset} = \frac{(T_2 - T_1) + (T_3 - T_4)}{2}$$

Under nominal conditions, both transit legs took 2.7 milliseconds. Due to path symmetry, the transit delays cancelled out perfectly, leaving a negligible offset of 0.01 milliseconds attributed to minor OS scheduling noise. However, when the traffic spike hit the request leg, the forward request took 13.5 milliseconds while the response remained at 2.7 milliseconds. This 10.8 milliseconds path asymmetry distorted the NTP math, forcing the computed offset to jump to 5.41 milliseconds as illustrated in Figure 3. This sudden spike highlights a mathematical degradation in computed clock accuracy rather than real physical clock drift, a phenomenon well documented in literature addressing routing link asymmetry [9] and queuing-induced delays [10].

On the execution side, the two paradigms displayed contrasting operational resilience. LF handled this fivefold latency spike seamlessly without crashing, dropping packets, or losing event ordering. Its runtime scheduler held the request event within its internal logical queue for exactly 13.5 milliseconds, delivering it to the server with absolute precision while preserving strict causal integrity across the system. Conversely, Embedded C suffered from an uncontrolled blocking window. Preventing the execution thread from hanging required substantial manual boilerplate, forcing the implementation to configure low-level socket timeouts and non-blocking I/O multiplexing loops. This requirement introduced a severe architectural penalty by tightly coupling high-level temporal logic with low-level network state machines.

Ultimately, while the path asymmetry compromised mathematical accuracy across both models, the reactor paradigm offered far superior resilience and synchronization. The LF scheduler absorbed real-world network fluctuations natively within its logical event queue, preserving application stability without requiring any manual asynchronous state-machine or timeout code.

B. Multi-Path Routing and Topology Variations

We explored diverse network connectivity configurations and server topologies to evaluate their effects on clock synchronization accuracy, execution safety, and timing precision. Introducing multi-hop network paths and concurrent server topologies exposed the client application to interleaved, asynchronous timestamp updates over asymmetric physical routes, causing fluctuating packet arrival windows and out-of-order delivery.

The empirical findings revealed a stark architectural divergence under these topological variations. LF natively preserved out-of-order execution safety without programmer intervention [11]; a packet taking a slower 3 milliseconds path through an alternate router did not block or corrupt the state of a packet taking a faster 0.5 millisecond path. The runtime framework automatically mapped these incoming streams back to their respective client ports based on logical tags, guaranteeing deterministic event delivery [5]. Conversely, the multi-threaded Embedded C architecture exhibited high vulnerability to data races and execution reordering. Handling multi-path non-deterministic arrival in C required implementing complex non-blocking state machines and dynamic array or buffer structures to track in-flight packets manually. Without careful memory safety, out-of-order packet arrivals caused variables to be overwritten or led to unhandled race conditions and thread jams, adding significant structural fragility and scheduling jitter that directly corrupted physical timestamp captures.

Ultimately, while complex topologies naturally compromise packet arrival order, the reactor paradigm handled message

Evaluation Metric	Imperative Embedded C	Reactive LF
Out-of-Order Transparency	Prone to data races and variable overwrites	Maintained natively via logical timestamps
Concurrency Infrastructure	Requires explicit low-level primitives (pthread_mutex)	Automated on-demand stream mapping to ports
Race Condition Guard	Demands manual, complex non-blocking buffers	Encapsulated isolated state inside reactors
Thread Synchronization	High risk of thread jams and race conditions	Zero manual structural adjustments needed

Table 3: Topological Resilience Metrics

reordering transparently via its native logical event queue. This ensured timing precision and thread safety without requiring the manual thread locking, dynamic buffers, or conditional barriers essential in an imperative architecture, as evaluated in Table 3.

C. Partial Network Failure

We introduced a conditional packet drop threshold within the network layer to evaluate system recovery and synchronization resilience under dynamic faults. Upon encountering a simulated 15% packet loss, the Network reactor immediately emitted a timestamped failure token via a dedicated output port at the exact logical tag t. Since LF reactions execute strictly on-demand, this output port remained silent during nominal operations. The dependent Client reactor's failure recovery handler was triggered if and only if an error token landed on its input port at logical time t. This event-driven design allowed the Client to execute lock-free recovery routines instantly at the precise logical tag where the drop occurred, ensuring that successful packet transmissions produced no tokens and incurred zero runtime overhead.

This approach contrasted sharply with the traditional imperative Embedded C socket architecture, where handling a silent network failure frequently trapped client threads within a blocking socket read call (*recvfrom*). Avoiding indefinite thread freezes forced the implementation to rely on low-level primitives for managing socket options, non-blocking polling loops, and complex asynchronous state machines. Furthermore, explicitly signaling errors across threads in C relied on shared memory or global state flags, which introduced significant risks of data races, variable overwrites, and thread starvation. Eliminating these risks required wrapping the state tracking in heavy synchronization primitives like *pthread_mutex*, adding severe scheduling jitter that directly degraded physical timestamp captures and corrupted clock calculations.

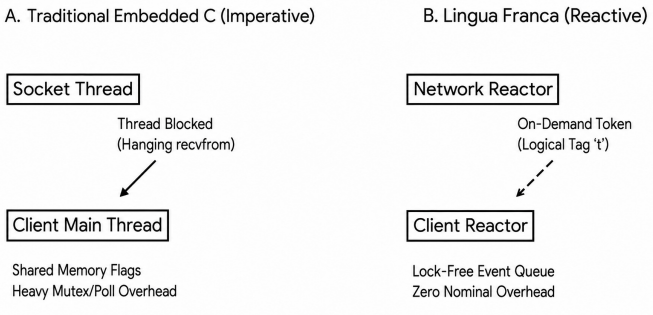


Figure 4: Architectural Comparison of Fault Signaling

Ultimately, while partial network failures disrupted packet flow across both models, the reactor paradigm handled dynamic fault recovery without introducing unhandled blocking states or architectural pollution. The LF runtime scheduler abstracted the thread-synchronization layer entirely, delivering the fault notification as an isolated, lock-free event. This permitted the client reactor to process network faults deterministically and reset its state without relying on shared memory, risking thread deadlocks, or imposing a runtime penalty during nominal operation, as outlined in Figure 4.

VI. Architectural Implications

To synthesize the empirical findings and development metrics, this section contextualizes the broader architectural implications of transitioning from imperative to reactive paradigms. We examine how coordinating software execution with real-world time impacts program complexity, analyze the cognitive trade-offs faced by system designers, and delineate the specific operational domains where each synchronization framework is most viable.

A. Infrastructure, Cognitive Load, and Trade-offs

The empirical findings from our network variability stress tests validate the structural trade-offs summarized in Table 4. In the traditional Embedded C paradigm, the programmer must explicitly reason about low-level memory management, event ordering, and edge cases like queue overflow or thread starvation. Under network variability, this manual approach requires substantial boilerplate code to handle socket timeouts and non-blocking I/O multiplexing, making the codebase highly susceptible to state machine fragmentation, race conditions, and non-deterministic operating system scheduling noise.

In contrast, the LF developer primarily reasons about the causality graph, defining precisely "what reacts to what" through ports and triggers. The underlying runtime framework automatically handles network variability, ensuring deterministic reaction ordering, out-of-order transparency, and lock-free fault recovery. This architectural offloading represents a substantial reduction in both cognitive burden and the overall error surface area [12] during distributed system development.

B. Applicability Domains

Embedded C remains indispensable for deeply resource-constrained domains, such as bare-metal microcontrollers and battery-powered IoT devices, where minimizing the runtime memory footprint is the absolute priority.

Conversely, Lingua Franca excels in complex Cyber-Physical Systems where correctness, verifiability, and rapid development cycles are paramount. While the baseline memory

Paradigm Trade-off	Embedded C	LF
Core Strengths	Guarantees minimal resource requirements (ideal for IoT/MCU) and direct hardware access	Ensures determinism via formal time semantics and native concurrency enabling superior abstraction
Core Weaknesses	Difficult to verify correctness/ concurrency; highly susceptible to race conditions and scheduling jitter	Higher runtime abstraction layer overhead; less suitable for deeply constrained devices
System Scalability	Manual synchronization limits system expansion; increases verification costs	Scaled easily by adding reactors and ports; the runtime automatically manages the scheduling graph

Table 4: Paradigm Trade-offs and Scalability

overhead of the LF runtime may appear prohibitive for tiny hardware targets, for systems with sufficient resource margins, the dramatic reduction in software architectural complexity often justifies the abstraction cost.

VII. Conclusion and Future Work

Embedded C offers minimal resource utilization and direct hardware control but requires manual, error-prone management of timing, concurrency, and component interaction. It remains the preferred choice for deeply resource-constrained, production-deployed systems. Conversely, Lingua Franca provides superior abstraction with formal time semantics, safe concurrency, and component reusability.

This study demonstrates that the distinction between these paradigms is not one of expressive capability, but of semantic abstraction. Lingua Franca internalizes a substantial temporal execution infrastructure — an estimated 8,000 lines of manual C code that must otherwise be constructed manually by the programmer. Our empirical evaluations under network variability validate that this abstraction directly translates to operational resilience; by leveraging native logical tag ordering and lock-free fault signaling, the reactor runtime successfully absorbed severe path asymmetry, out-of-order packet delivery, and partial network failures natively within its logical event queue, whereas the imperative Embedded C alternative suffered from severe thread blocking, state fragmentation, and scheduling jitter.

As time-synchronized Cyber-Physical Systems scale in complexity, the reduction of software engineering overhead and accidental complexity becomes increasingly valuable, offering significant structural advantages in distributed implementations where correctness, determinism, and the precise alignment of software execution logic with real-world time are paramount.

While this study transitioned the evaluation from a localized simulation to an empirical network environment modeling bandwidth throttling, multi-path routing, and stochastic packet loss, the execution still occurred within a single physical host framework. Future research will extend this methodology to a physically distributed environment across a multi-node hardware cluster (e.g., Raspberry Pi single-board computers) connected over an active physical Ethernet topology.

This transition will allow us to evaluate how the LF runtime infrastructure (RTI) maintains a shared logical schedule [13] when the physical hardware clocks on separate nodes naturally drift over time. Resolving these physical drifts under decentralized runtime conditions will require the integration and evaluation of hardware-level precision clock synchronization alternatives, such as the Precision Time Protocol (IEEE 1588 PTP) [14], [15], to supplement the software-level scheduling loop.

References

[1] E. A. Lee and S. A. Seshia, *Introduction to Embedded Systems: A Cyber-Physical Systems Approach*, 2nd ed. MIT Press, 2017.

[2] E. A. Lee, "The problem with threads," *IEEE Computer*, vol. 39, no. 5, pp. 33–42, May 2006.

[3] D. L. Mills, "Internet time synchronization: The network time protocol," *IEEE Transactions on Communications*, vol. 39, no. 10, pp. 1482–1493, 1991.

[4] D. L. Mills, J. Martin, J. Burbank, and W. Kasch, "Network time protocol version 4: Protocol and algorithms specification," IETF, RFC 5905, June 2010.

[5] E. A. Lee, M. Lohstroh, C. Menard, S. Barrows, and I. Castrillon, "Toward a lingua franca for deterministic reactive systems," *ACM Transactions on Embedded Computing Systems (TECS)*, vol. 20, no. 4, 2021.

[6] G. Berry and G. Gonthier, "The esterel synchronous programming language: Design, semantics, implementation," *Science of Computer Programming*, vol. 19, no. 2, pp. 87–152, 1992.

[7] C. Menard *et al.*, "High-performance deterministic concurrency using lingua franca," *arXiv preprint arXiv:2301.02444*, 2023.

[8] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.

[9] J. Kim, H. Lee, and S. C. Kim, "Clock offset estimation for systems with asymmetric packet delays," *IEEE/ACM Transactions on Networking*, vol. 31, no. 2, pp. 640–652, 2023.

[10] Z. Chaloupka, N. Alsindi, and J. Aweya, "Clock synchronization over communication paths with queue-induced delay asymmetries," *IEEE Communications Letters*, vol. 18, no. 9, pp. 1531–1534, 2014.

[11] S. Lin *et al.*, "Towards building verifiable cps using lingua franca," *ACM Transactions on Embedded Computing Systems*, vol. 22, no. 5, pp. 1–24, 2023.

[12] C. Menard, S. Bateni, M. Lohstroh, and E. A. Lee, "Deterministic real-time coordination for distributed reactive systems," *IEEE Embedded Systems Letters*, vol. 14, no. 3, pp. 115–118, Sept 2022.

[13] B. Jun, E. A. Lee, M. Lohstroh, and H. Kim, "Efficient coordination for distributed discrete-event systems," *arXiv preprint arXiv:2410.06454*, 2024.

[14] Z. Tang and G. Yan, "Research on enhancing the time synchronization accuracy of ieee 1588 based on asymmetric delay prediction," in *ACM Proceedings on Computer Network and Cloud Computing*, 2025, pp. 52–56.

[15] *IEEE Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems*, IEEE Std. IEEE Std 1588-2019 (Revision of IEEE Std 1588-2008), 2020.